

Lesson 10

React Router & Client-Side Routing

Code the Dream — React

Today's Session

EXPLORE

- SPAs vs. traditional multi-page sites
- What React Router solves
- BrowserRouter, Routes, Route
- Shared layout — what stays, what changes
- Link VS. NavLink
- Protected routes, dynamic routes, catch-all routes
- `useNavigate` and `useLocation`

APPLY

- Wire up 5 routes with a shared layout
- Add active-styled `NavLink` navigation
- Read dynamic URL params with `useParams`
- Use `useNavigate` for programmatic nav and `useLocation` for the 404 page

By the end: implement client-side routing so your SPA has real URLs, working back/forward buttons, and shareable links.

SPA vs. Traditional Navigation

Traditional website

- Each page is a separate URL + HTML file
- Browser fetches a new document on every click
- Back/forward buttons work automatically
- URLs are shareable and bookmarkable

React SPA (without routing)

- One HTML file — JS swaps the content
- URL never changes while navigating
- Back button may exit the app entirely
- Can't link someone to a specific view

React Router bridges this gap — it updates the URL and the browser history stack without ever fetching a new HTML document from the server.

Setup — Install and Wrap with BrowserRouter

Install: `npm install react-router`

Wrap your app in `<BrowserRouter>` in `main.jsx` — this provides navigation and location context to everything inside:

```
import { BrowserRouter } from 'react-router';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

`BrowserRouter` uses the browser's History API under the hood. It's similar to the context providers from last lesson — wrap once at the top, access anywhere inside.

Routes and Route — Pattern Matching

`<Routes>` inspects the current URL and renders the first `<Route>` whose path matches.

```
<Routes>
  <Route path="/" element={<Shop />} />
  <Route path="/checkout" element={<Checkout />} />
  <Route path="/account" element={<Account />} />
  <Route path="/products/:id" element={<ProductDetails />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

- Only **one** Route renders at a time — whichever path matches the URL
- `:id` is a dynamic segment — React Router extracts its value for use in the component
- `path="*" is a catch-all — matches any URL that didn't match above it`

Shared Layout — What Stays, What Changes

Place Header and Footer outside `<Routes>` so they render on every page. Only `<main>` changes.

```
function App() {  
  return (  
    <>  
      <Header />  
      <main>  
        <Routes>  
          <Route path="/" element={<Shop />} />  
          <Route path="/checkout" element={<Checkout />} />  
          <Route path="*" element={<NotFound />} />  
        </Routes>  
      </main>  
      <Footer />  
    </>  
  );  
}
```

Every "page" in the app is just a different component rendered inside `<main>` — the rest of the shell stays constant.

Link VS. NavLink

Both navigate without a page reload. Use `<a href>` only for external links.

```
// Link – standard client-side navigation
<Link to="/checkout">Go to Checkout</Link>

// Use <a> for links leaving the app
<a href="https://codethedream.org">Code the Dream</a>

// NavLink – adds "active" class when its route matches the URL
<NavLink to="/" end>Home</NavLink>
<NavLink to="/checkout">Checkout</NavLink>

/* CSS – style the active link */
a.active { font-weight: bold; color: #FF5C35; }
```

The `end` prop on the Home `NavLink` prevents it from matching as active on every route (since every URL starts with `/`).

Protected Routes

Conditionally include a `<Route>` in the JSX — if the condition is false, the route simply doesn't exist.

```
<Routes>
  <Route path="/" element={<Shop />} />
  <Route path="/checkout" element={<Checkout />} />
  {user.id && (
    <Route path="/account" element={<Account user={user} />} />
  )}
  <Route path="*" element={<NotFound />} />
</Routes>
```

When the user is logged out, `/account` matches no Route → the `*` catch-all renders `<NotFound>` instead.

This is simpler than redirecting — the route doesn't exist at all when the condition is false.

Check for Understanding

A user is on `/checkout` and clicks a button labeled "Back to Shop." Which is the best implementation?

- A) `<a href="/">Back to Shop</a>`
- B) `<Link to="/">Back to Shop</Link>`
- C) `<NavLink to="/">Back to Shop</NavLink>`
- D) `navigate(-1)` inside an `onClick` handler

Dynamic Routes and useParams

Add `:paramName` to a path to capture that URL segment as a variable.

```
<Route path="/products/:id" element={<ProductDetails />} />
```

Inside the component, `useParams()` returns an object with all captured segments:

```
import { useParams } from 'react-router';

function ProductDetails({ products }) {
  const { id } = useParams(); // value from the URL – always a string

  const product = products.find(p => p.id === id);
  if (!product) return <p>Product not found.</p>;

  return <h2>{product.name}</h2>;
}
```

`useParams` always returns strings. Convert to a number with `Number(id)` or `parseInt(id)` if your IDs are numeric.

Catch-All Route — 404 Pages

`path="*" matches any URL that didn't match any Route above it. Always put it last.`

```
<Route path="*" element={<NotFound />} />
```

Without a catch-all, unmatched URLs render nothing — a blank screen with no feedback for the user.

```
function NotFound() {  
  return (  
    <>  
      <h2>404 – Page not found</h2>  
      <Link to="/">Go home</Link>  
    </>  
  );  
}
```

A catch-all route is essential. Users mistype URLs, old links go stale, and bots probe random paths — always handle the unmatched case.

useNavigate — Programmatic Navigation

Use `useNavigate` when you need to navigate in response to an event — not a user clicking a link.

```
import { useNavigate } from 'react-router';

function Cart({ handleCloseCart }) {
  const navigate = useNavigate();

  function handleCheckout() {
    handleCloseCart();
    navigate('/checkout'); // navigate to a path
  }
}
```

Pass a **delta number** to navigate through history instead:

- `navigate(-1)` — same as the browser's back button
- `navigate(1)` — same as the browser's forward button

useLocation — Reading the Current URL

`useLocation` returns an object describing the current route — useful for displaying the active path or reading query strings.

```
import { useLocation, Link } from 'react-router';

function NotFound() {
  const { pathname } = useLocation();
  return (
    <>
      <h2>404 – Page not found</h2>
      <p><code>{pathname}</code> does not exist.</p>
      <Link to="/">Go home</Link>
    </>
  );
}
```

useLocation — Continued

The location object also includes:

- `location.search` — the query string, e.g. `?sort=price`
- `location.hash` — the fragment, e.g. `#reviews`

Now: Apply

Four exercises. One routing concept each.

This Week's Exercise — Routing Demo App

Open `src/exercises/lesson-10/studentWork.jsx`. You'll build a small app where Header and Footer always render and `<main>` changes with the URL.

Exercise	Concept
1	Routes + Route + shared layout + protected route
2	NavLink with active styling
3	Dynamic route + useParams
4	useNavigate + useLocation + 404

Pre-built page components are in `pages/` — Home, Checkout, ProductDetails, Account, NotFound. You wire them up, you don't build them from scratch.

Exercise 1 — Routes + Protected Route

In `studentWork.jsx`, implement the full route table inside `<main>`:

```
<main>
  <Routes>
    <Route path="/" element={<Home products={products} />} />
    <Route path="/checkout" element={<Checkout />} />
    <Route path="/products/:id" element={<ProductDetails />} />
    {isLoggedIn && (
      <Route path="/account" element={<Account />} />
    )}
    <Route path="*" element={<NotFound />} />
  </Routes>
</main>
```

Import `products` from the `data/` directory, create a `products` state variable, and pass it to `<Home>`.

Test: Type `/account` in the URL bar while logged out — `NotFound` should render, not `Account`.18

Exercise 2 — NavLink + Active Styling

In `components/Header.jsx`, replace any plain links with `NavLink`:

```
<NavLink to="/" end>Home</NavLink>  
<NavLink to="/checkout">Checkout</NavLink>  
{isLoggedIn && <NavLink to="/account">Account</NavLink>}
```

Add CSS that styles the active link differently:

```
a.active {  
  font-weight: bold;  
  text-decoration: underline;  
}
```

Test: Navigate between pages — the active link should change. The Account link should disappear when logged out.

Exercise 3 — Dynamic Routes and useParams

In `pages/Home.jsx` — add a link from each product card to its detail page:

```
<Link to={`/${products}/${product.id}`}>{product.name}</Link>
```

In `pages/ProductDetails.jsx` — read the URL param and look up the product:

```
const { id } = useParams();  
const product = products.find(p => p.id === id);  
  
if (!product) return <p>Product not found.</p>;
```

Add a `<Link to="/">Back to Shop</Link>` at the bottom of `ProductDetails`.

Test: Click a product → URL changes to `/products/[id]` → correct product renders. Type a bad ID → "not found" message appears.

Exercise 4 — useNavigate + useLocation

In `pages/Checkout.jsx` — add programmatic navigation buttons:

```
const navigate = useNavigate();

<button onClick={() => navigate('/')}>Go Home</button>
<button onClick={() => navigate(-1)}>Back</button>
```

In `pages/NotFound.jsx` — display the invalid path:

```
const { pathname } = useLocation();

<h2>404 – Page not found</h2>
<p><code>{pathname}</code> does not exist.</p>
<Link to="/">Go Home</Link>
```

Test: On Checkout, "Back" should go to wherever you came from. On a bad URL like `/xyz`, the 404 page should show `/xyz`.

Wrap-Up

This week you learned:

- React SPAs need client-side routing to support real URLs, browser history, and shareable links
- BrowserRouter → Routes → Route is the setup hierarchy
- Header and Footer outside <Routes> creates a persistent shell; only <main> changes
- Link navigates without reload; NavLink also highlights the active route
- Dynamic paths use :paramName — read with useParams() (always returns strings)
- useNavigate navigates programmatically; useLocation reads the current URL

Before next session: Complete the weekly assignment — add React Router to your todo app, create separate pages for the todo list and account, and protect the todo page behind authentication. Submit your PR link in CTD Learns. Post questions in **#[class-name]-discussion** on Slack.